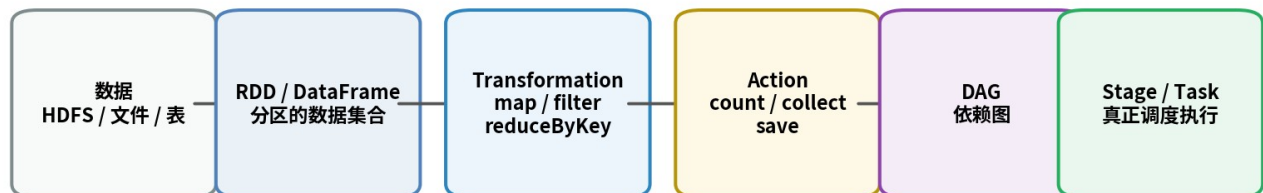


大数据处理技术：从 MapReduce 到 Spark

第十一讲：为什么需要 Spark

RDD • Transformation • Action • Lazy Evaluation • DAG • Stage • Task

Spark 第一层心智模型



你写的是“数据要怎样变换”；Spark 先记录依赖关系，等 Action 出现后再生成执行计划。

本讲目标：不是背 Spark API，而是回答“Spark 为什么出现，以及它到底怎样把一串数据操作变成分布式执行任务”。

一、先回答最重要的问题：MapReduce 已经能处理海量数据，为什么还要 Spark?

前十讲我们已经建立了 MapReduce 的完整闭环：HDFS 把数据分块，Mapper 并行处理，Shuffle 按 Key 重新组织，Reducer 聚合，YARN 负责资源与任务调度。它很可靠，但它的执行模型比较“笨重”。

MapReduce 的典型优势是：一次性批处理、计算逻辑清晰、失败可重试、适合海量数据。但当问题需要反复使用同一批中间数据时，传统 MapReduce 会产生大量磁盘 I/O 和 Job 启动开销。

关键矛盾：MapReduce 的问题不是“不会算”，而是很多现代工作负载需要反复迭代、交互式分析或多步骤流水线；每一步都启动一个 Job、频繁落盘，会非常低效。

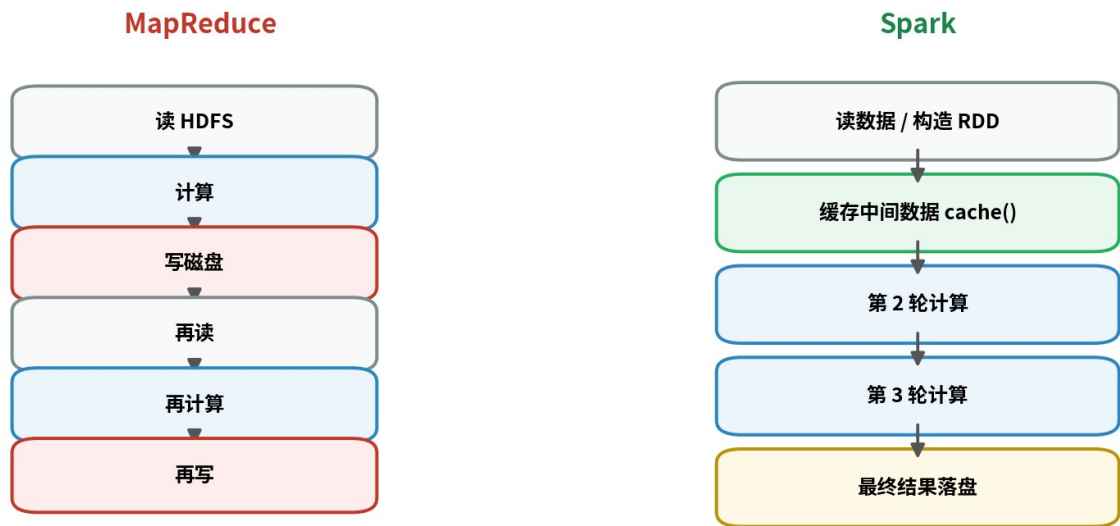
场景	MapReduce 的感受	Spark 的改进方向
一次日志统计	很合适	也可以做，但优势不一定明显
十几步 ETL 流水线	多个 Job 串联，繁琐	一个 DAG 统一规划
PageRank / 机器学习迭代	反复读写磁盘	缓存中间数据后反复复用
交互式分析	Job 启动成本较明显	更适合连续执行多步操作
流式/统一数据处理生态	原生 MapReduce 能力有限	Spark 生态提供更统一接口

二、一个例子：为什么“迭代算法”会让 MapReduce 很难受

假设有一个算法需要对同一份数据计算 20 轮。第 1 轮产生中间结果，第 2 轮继续使用第 1 轮结果……这在 PageRank、聚类、机器学习优化等任务中很常见。

在经典 MapReduce 思路下，经常会出现：第 1 个 Job 写结果到磁盘/HDFS -> 第 2 个 Job 再读 -> 再写 -> 第 3 个 Job 再读。真正浪费时间的往往不是加法和乘法，而是磁盘和网络。

迭代计算：MapReduce 与 Spark 的关键差别



核心不是“永远不落盘”，而是：Spark 可以尽量复用内存中的中间结果，减少反复磁盘 I/O。

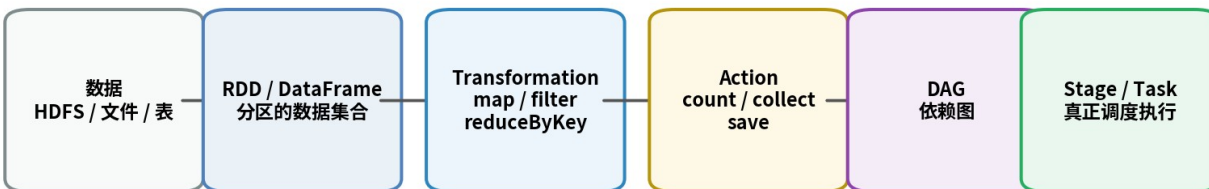
不要理解过头：Spark 不是“所有数据永远放内存，也永远不落盘”。内存不够、发生 Shuffle、缓存策略变化时依然可能使用磁盘。它的核心是：允许中间数据被缓存和复用，并用更灵活的执行引擎组织整个计算图。

三、Spark 最核心的转变：从“一个 Job 一个 Map/Reduce 模板”变成“数据依赖图”

MapReduce 的编程结构比较固定：Map -> Shuffle -> Reduce。Spark 不要求每个任务都硬塞成这一套。你可以写一串数据变换：读取 -> 过滤 -> 映射 -> 分组 -> 聚合 -> 再映射 -> 保存。

Spark 先记录这些操作之间的依赖关系，形成一个有向无环图（DAG, Directed Acyclic Graph），等真正需要结果时，再把 DAG 切成 Stage，并把每个 Stage 拆成多个 Task 并行执行。

Spark 第一层心智模型



你写的是“数据要怎样变换”；Spark 先记录依赖关系，等 Action 出现后再生成执行计划。

MapReduce	Spark 中更一般化的对应思想
MapTask	某个 Stage 中处理一个分区的 Task
ReduceTask	发生 Shuffle 后下游 Stage 中的一类 Task
一个 Job 的固定 Map/Reduce 两阶段	一个 Job 可以包含多个 Stage
多个 Job 串接复杂流程	一个 DAG 可以表达多步依赖
中间结果常通过磁盘衔接	可以 cache/persist 复用中间数据

四、RDD 到底是什么？不要把它理解成“一个巨大的 Python 列表”

RDD 全称 Resilient Distributed Dataset，中文通常译为“弹性分布式数据集”。这个名字里每个词都有含义：

- Dataset：逻辑上是一批数据，例如日志行、订单、键值对。

从 MapReduce 到 Spark：为什么需要 Spark，RDD / DAG / Stage / Task 入门

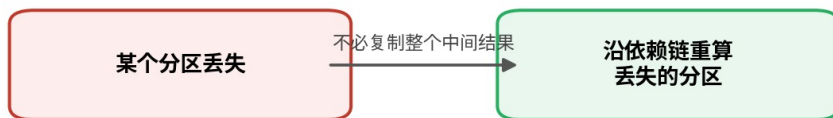
- **Distributed**: 数据被切成多个 Partition，分布在集群不同节点上并行处理。
- **Resilient**: 某个分区丢失后，Spark 可以根据依赖关系（Lineage）重新计算，而不是必须给每份中间结果做昂贵的多副本。

最重要的一点：RDD 是“逻辑数据集 + 分区 + 依赖关系”的抽象。Driver 端通常并没有真的把全体数据装进一个 Python List。

RDD Lineage : Spark 为什么可以“重新算回来”



Lineage = “这个数据是怎么从前面的数据一步步推导出来的”



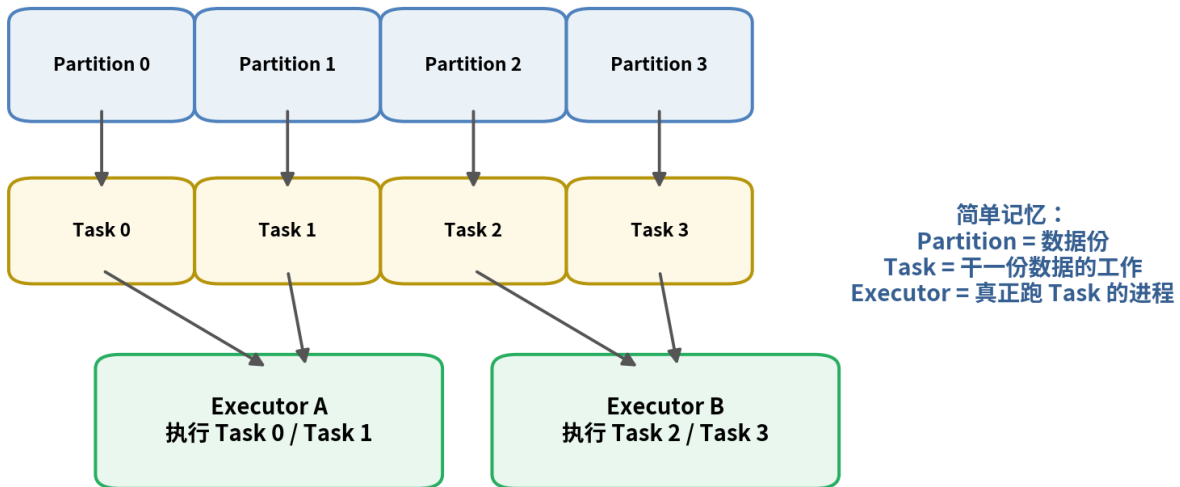
与 HDFS 副本的区别: HDFS 更偏向“存储容错”：数据块保存多份副本。RDD 的 Lineage 更偏向“计算容错”：中间分区丢了，可以沿着计算链重新生成。二者解决的是不同层次的问题。

五、Partition：Spark 并行计算真正的基本数据单位

Spark 中一个 RDD/DataFrame 会被切成多个 Partition。可以先把 Partition 理解成“需要分别处理的若干份数据”。同一个分区内部通常由一个 Task 顺序处理，而多个分区可以被多个 Task 并行处理。

概念	最简单理解	常见关系
Partition	一份数据	决定可以拆出多少并行任务
Task	处理一份 Partition 的一次工作	一个 Stage 往往有很多 Task
Executor	Worker 节点上的执行进程	同时/轮流运行多个 Task
Stage	一组可以连续流水执行的算子	通常在 Shuffle 边界处被切开
Job	一次 Action 触发的一整套计算	可以包含多个 Stage

Partition、Task、Executor 的关系



一个很重要的迁移：你在 MapReduce 中已经学过“InputSplit 决定 Mapper 并行度”的思想。到 Spark 里，要把注意力迁移到 Partition：数据如何分区，直接影响 Task 数、并行度、Shuffle 和数据倾斜。

六、Transformation 和 Action：Spark API 的第一条分界线

Spark 的操作可以先粗略分成两类。Transformation（转换）描述“数据要怎样变”；Action（动作）要求“现在给我结果”。

类型	典型操作	作用	是否立即触发真正计算
Transformation	map, filter, flatMap, reduceByKey	产生一个新的逻辑数据集	通常不立即
Action	count, collect, take, saveAsTextFile	返回结果或写出结果	通常会触发 Job

一条最小 PySpark 数据链

```

# PySpark 概念示例
lines = sc.textFile("hdfs:///data/logs")    # 得到一个 RDD
paid = lines.filter(lambda x: "PAID" in x)  # Transformation
pairs = paid.map(lambda x: (x.split(",")[3], 1)) # Transformation
counts = pairs.reduceByKey(lambda a, b: a + b) # Transformation

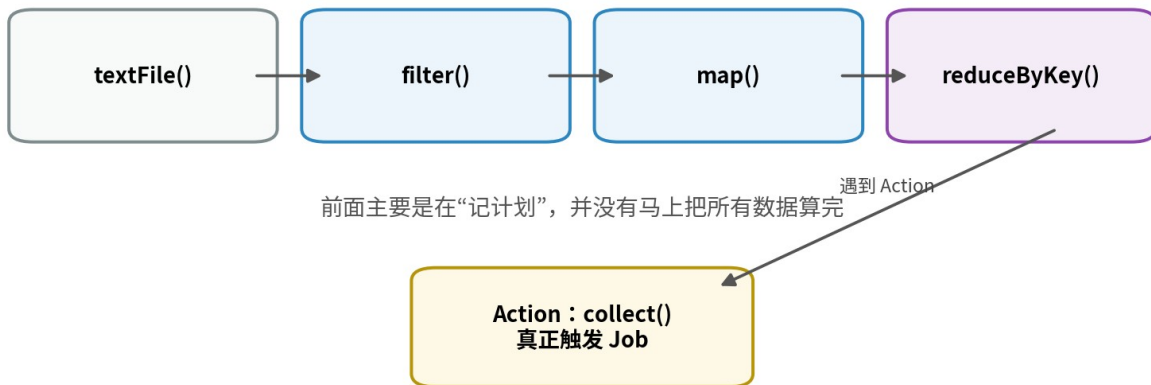
print(counts.count()) # Action: 到这里才真正需要算出结果
    
```

初学者误区：写完 filter() 并不代表集群已经把所有数据过滤完了。Spark 更多是在 Driver 端构造一个“如何计算”的计划；直到 Action 需要结果，执行才被真正触发。

七、Lazy Evaluation：为什么要“偷懒”？

Lazy Evaluation 通常翻译为“惰性求值”。这个“懒”不是性能差，而是故意不急着执行。因为如果每写一行 Transformation 就立刻运行，Spark 看不到后面的操作，也就很难统一优化和调度。

Lazy Evaluation：为什么 Transformation 不立即计算



好处：Spark 能看到整条依赖链，再决定如何切 Stage、合并操作、安排执行。

一个直观类比：你去仓库取货。不是每收到一个动作“走到 A 区、拿苹果、再走 B 区、拿牛奶”就单独派一次车，而是先收集完整清单，然后规划一条更合理的路线。

Lazy Evaluation 还带来一个现实效果：如果你只是定义了一个很复杂的 RDD，但从未调用 Action，那么 Spark 可能根本不会真正把这条计算链跑完。

八、DAG：Spark 为什么比 MapReduce 的“两阶段模板”更灵活

DAG 是有向无环图。每个节点可以理解为一个数据集/计算步骤，边表示依赖关系。“无环”意味着数据流总体向前推进，不会形成无限回路。迭代算法不是靠 DAG 自己形成环，而是由程序逻辑多次触发/复用数据。

Spark 会根据 Transformation 之间的依赖构造 DAG。一个非常重要的区分是：窄依赖（Narrow Dependency）通常允许上游一个分区直接供下游少数分区使用；宽依赖（Wide Dependency）往往意味着数据需要跨分区重新分布，也就是 Shuffle。

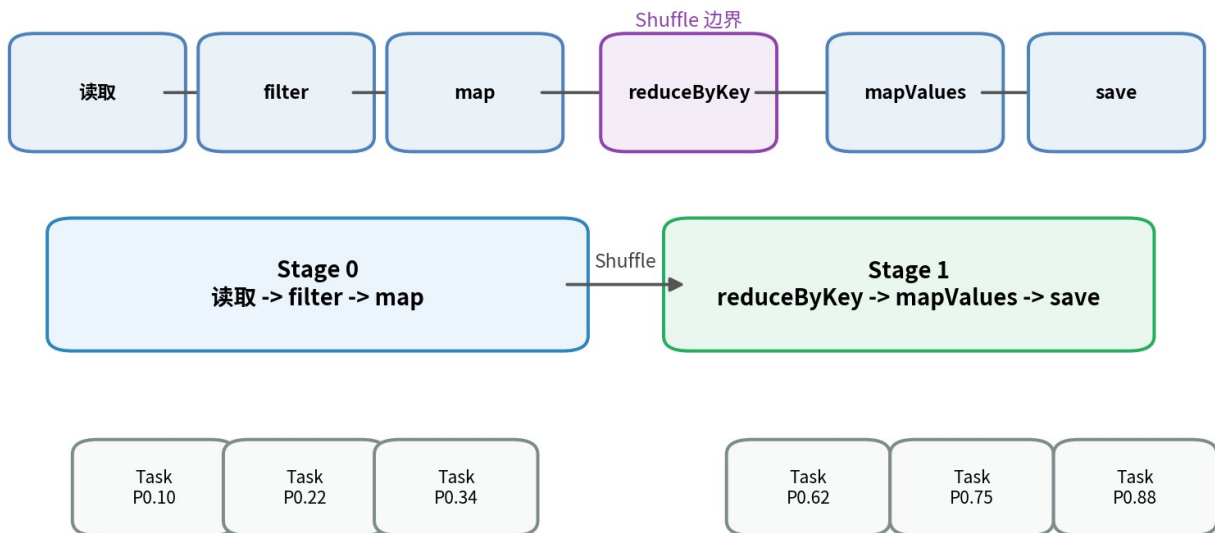
依赖类型	直观特征	典型操作	对执行计划的影响
窄依赖	下游一个分区只依赖少量上游分区	map, filter	容易流水执行在同一 Stage
宽依赖	下游分区要从许多上游分区取数据	reduceByKey, groupByKey	通常产生 Shuffle，形成 Stage 边界

与 MapReduce 的联系：你前面学的 Shuffle 没有消失。Spark 依然非常重视 Shuffle，只是它把 Shuffle 放进更一般的 DAG 执行模型中。很多 Spark 性能问题，本质仍然是“分区、Shuffle、倾斜、网络和磁盘”。

九、Stage：为什么 Spark 不把整个 DAG 一口气当成一个任务？

因为有些操作可以在同一份数据上连续流水执行，而有些操作必须先把数据重新按 Key 分区并跨网络交换。Spark 通常会以 Shuffle 依赖为重要边界，把一个 Job 切成多个 Stage。

DAG、Stage、Task：Spark 到底怎么执行



一个 Stage 内通常有多个 Task；Task 数量通常与该 Stage 要处理的分区数对应。

例如：textFile -> filter -> map 可以在每个分区本地连续做，不需要跨节点重新组织数据；reduceByKey 需要按 Key 重分区，就形成 Shuffle，Shuffle 前后通常属于不同 Stage。

记忆法：Job 是“一次要结果的完整任务”；Stage 是“中间不用跨分区重排、可以连续跑的一段”；Task 是“某个 Stage 里处理一个 Partition 的具体工作”。

十、Task：Spark 最终还是要把抽象图变成具体工作

DAG、Stage 都是计划层概念。真正让 CPU 干活的是 Task。一个 Stage 有 N 个分区，通常就需要 N 个 Task 去处理这些分区；这些 Task 会被调度到不同 Executor 上执行。

因此如果你看到 Spark UI 中某个 Stage 有 200 个 Tasks，可以先理解为：这一阶段要处理大约 200 份分区数据。

现象	底层可能意味着什么
Task 太少	并行度可能不足，CPU 利用率不高
Task 太多且每个很小	调度开销可能变大，小文件/小分区问题
只有少数 Task 特别慢	可能存在数据倾斜、慢节点或 GC 问题
Shuffle Read/Write 很大	宽依赖导致大量网络和磁盘 I/O

十一、用 WordCount 比较：MapReduce 和 Spark 到底差在哪里

同一个 WordCount，业务逻辑没有变：拆词 -> (word,1) -> 相同 word 聚合。但编程模型不同。

步骤	MapReduce	Spark
读取	InputFormat / RecordReader	textFile
拆词	Mapper	flatMap
构造键值对	Mapper context.write	map
按 Key 聚合	Shuffle + Reducer	reduceByKey (内部仍有 Shuffle)
输出	OutputFormat/HDFS	saveAsTextFile 等
复杂多步骤	通常多个 Job 串联	可在一个 DAG 中描述

Spark 版本的 WordCount

```
# PySpark WordCount (概念版)
text = sc.textFile("hdfs:///input")
words = text.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs:///output")
```

代码短不代表没有底层机制。reduceByKey 背后依然可能发生 Partition、Shuffle、网络传输、排序/聚合、Task 调度。Spark 只是把这些机制封装得更一般、更灵活。

十二、Spark 是不是“完全替代 Hadoop”？

不是。这里要把几个层次分开：Hadoop 不等于 MapReduce。Hadoop 生态中的 HDFS 仍然可以作为 Spark 的数据存储；YARN 也可以作为 Spark 的资源管理器。Spark 主要替代的是很多原来用 MapReduce 执行引擎完成的计算。

组件/概念	主要职责	Spark 出现后是否仍可使用
HDFS	分布式存储	可以，Spark 可直接读写 HDFS
YARN	资源管理/调度	可以，Spark 可运行在 YARN 上
MapReduce	批处理计算引擎/模型	很多场景被 Spark 等替代
Spark	通用分布式计算引擎	可与 HDFS/YARN 组合

关键理解：“Spark 替代 Hadoop”这个说法不够准确。更准确的是：Spark 在很多场景中替代 Hadoop MapReduce 作为计算引擎，但它完全可以继续使用 Hadoop 的 HDFS 和 YARN。

十三、Spark 一定比 MapReduce 快吗？

不能简单说“一定”。如果只是一次性扫描超大文件、做非常简单的批处理，而且中间结果不复用，那么 Spark 的内存优势可能并不明显。Spark 的优势在多步骤 DAG、迭代计算、缓存复用、交互式分析和统一 API 上更明显。

情况	更值得关注的因素
一次性简单批处理	磁盘吞吐、网络、数据本地性，MapReduce 仍能胜任
反复迭代	缓存复用，中间结果避免反复落盘
多步 ETL	DAG 统一组织、避免手工串多个 Job
大量 Shuffle	Spark 也会慢，仍需优化分区/倾斜/序列化
内存不足	可能 Spill 到磁盘，不能把 Spark 等同于“纯内存计算”

十四、把前十讲的 MapReduce 知识迁移到 Spark

你已经会的 MapReduce 思想	迁移到 Spark 后继续有用
InputSplit / 并行输入	Partition / Input Partitioning
Mapper 并行处理	Task 对 Partition 并行计算
Partitioner	Hash/Range Partitioning，自定义分区
Shuffle	宽依赖、Stage 边界、Shuffle Read/Write
Reducer 数据倾斜	Spark 中同样有 Skew / Straggler
Combiner / 本地聚合	reduceByKey 等会尽量做 map-side combine
YARN Container	Spark on YARN 中 Driver/Executor 申请资源
Task retry / 容错	Task 重算 + RDD Lineage 等机制

为什么先学 MapReduce 很值：如果直接学 Spark，很容易只会写 DataFrame API，却看不懂为什么某个 groupBy 突然很慢。你已经掌握 Shuffle、Partition、倾斜和容错，再学 Spark 会直接看到这些 API 背后的分布式机制。

十五、初学者最容易产生的 12 个疑问

- 1. Spark 是数据库吗？** 不是。它是分布式计算引擎，但可以通过 Spark SQL 处理结构化数据。
- 2. RDD 是存储在一台机器内存里的大数组吗？** 不是。它是分区式逻辑数据集，数据分散在多个节点/分区。
- 3. Spark 所有数据都在内存吗？** 不是。数据可能来自 HDFS，也可能缓存到内存；内存不足和 Shuffle 时也会用磁盘。
- 4. Transformation 为什么不马上执行？** Spark 要先构造依赖计划，Action 触发时再统一调度。
- 5. 一个 Action 一定对应一个 Job 吗？** 通常可以这样建立入门理解；复杂执行细节以后再精确展开。
- 6. 一个 Job 只有一个 Stage 吗？** 不一定。发生 Shuffle 时通常会切成多个 Stage。
- 7. 一个 Stage 只有一个 Task 吗？** 通常不是。Stage 中会为多个 Partition 启动多个 Task。
- 8. reduceByKey 还有 Shuffle 吗？** 有。Spark 没有消灭 Shuffle，它只是用更灵活的 DAG 管理它。
- 9. cache() 一调用数据就马上进内存吗？** 通常不是。cache 是声明缓存策略，仍需 Action 真正触发计算。
- 10. RDD 丢了一个分区怎么办？** 可以根据 Lineage 从上游数据重新计算丢失分区。

11. Spark 和 YARN 谁调度谁？ YARN 管集群资源；Spark 自己负责一个 Spark Application 内部的 DAG/Stage/Task 调度。

12. 以后是不是完全不用学 MapReduce？ API 可以少写，但 MapReduce 训练出的 Shuffle、分区、数据并行和容错思维仍然是基础。

十六、本讲练习：不要写代码，先画执行图

任务：有 1 TB 订单数据，要统计每个城市的 PAID 订单总金额。请先用 Spark 思维回答：

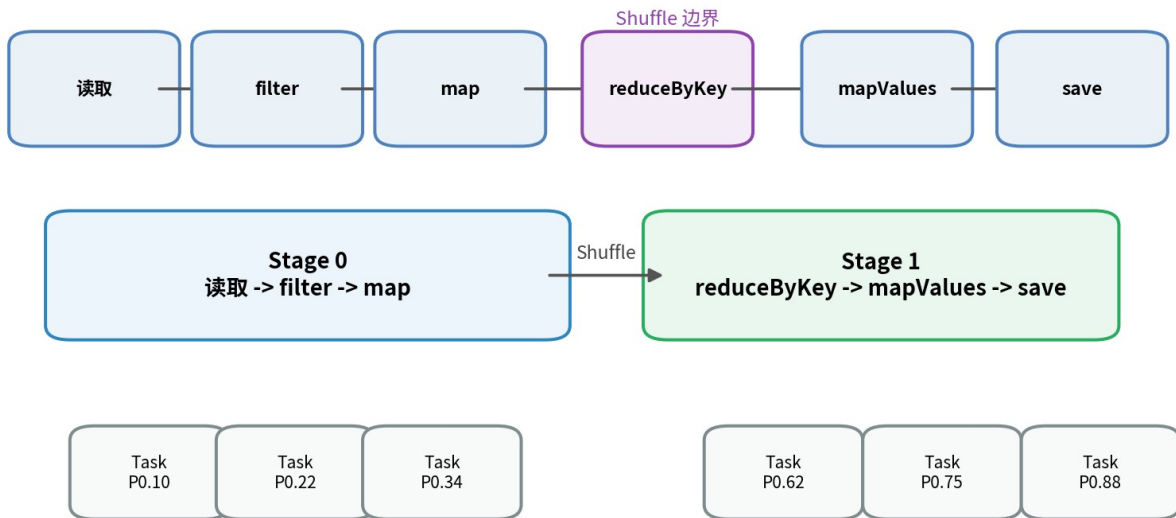
1. 原始数据大概会被切成什么？——多个 Partition。
2. filter(status == PAID) 属于 Transformation 还是 Action？
3. map(order -> (city, amount)) 是否需要 Shuffle？
4. reduceByKey(sum) 为什么会产生 Shuffle？
5. Shuffle 前后会形成几个 Stage（入门级判断）？
6. 每个 Stage 为什么又会有多个 Task？
7. 如果某个城市占全部订单 60%，可能出现什么问题？

参考思路：filter/map 是窄依赖型 Transformation；reduceByKey 需要把同一 city 的数据重新聚到一起，因此产生宽依赖/Shuffle，通常会形成 Stage 边界；热点城市可能造成某个分区特别大，形成数据倾斜和长尾 Task。

十七、这一讲只需要牢牢记住 7 句话

- Spark 出现的核心原因之一，是传统 MapReduce 在多步骤和迭代计算中频繁落盘、Job 串联成本较高。
- RDD 是“有分区、有依赖、可容错”的分布式数据集合抽象，不是单机 List。
- Transformation 主要描述“怎样变换数据”，Action 通常才触发真正计算。
- Lazy Evaluation 让 Spark 先看到整条计算链，再生成执行计划。
- DAG 是依赖图；Shuffle/宽依赖通常形成 Stage 边界。
- Stage 里面有很多 Task；Task 通常一份一份地处理 Partition。
- Spark 没有消灭 Shuffle、分区、倾斜和磁盘，它只是提供了比 MapReduce 更一般、更灵活的执行模型。

DAG、Stage、Task : Spark 到底怎么执行



一个 Stage 内通常有多个 Task ; Task 数量通常与该 Stage 要处理的分区数对应。

十八、下一讲预告：RDD 与 Spark 程序到底怎样运行

下一讲建议继续把本讲的抽象概念落地：从一个 PySpark 程序开始，详细讲 SparkSession / SparkContext、Driver、Executor、Worker、Job、Stage、Task、Partition 的完整运行关系，并实际追踪一次 WordCount 从代码到执行计划的全过程。

到那一步，我们再正式区分：Driver 是什么、Executor 是什么、Spark on YARN 时又怎样和 ResourceManager / NodeManager 对接。